

GDG McGill × CodeJam McGill

Intro to Front-End Development

From HTML to AI

Jackson Yu

What is front end?

Front end

What you see

```
team-finder.app
```

Navbar

Card | Card | Card

Button

Back end

What runs behind it

- Stores the data
- Runs the logic
- Handles login





The three building blocks



HTML

Structure



CSS

Style

JavaScript

Behavior

Live demo: a simple website

index.html

```
<!doctype html>
<html>
<head>
  <title>My first website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello, hackathon!</h1>
  <p>This page has HTML, CSS and JavaScrip
  <button id="btn">Clicked 0 times</buttor

  <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
  font-family: sans-serif;
  text-align: center;
  padding-top: 100px;
}

button {
  background: royalblue;
  color: white;
  border: none;
  border-radius: 8px;
  padding: 12px 24px;
  font-size: 18px;
}
```

script.js

```
let count = 0;
const button = document.getElementById("bt

button.addEventListener("click", () => {
  count = count + 1;
  button.textContent = "Clicked " + count
});
```

Why the old way got painful

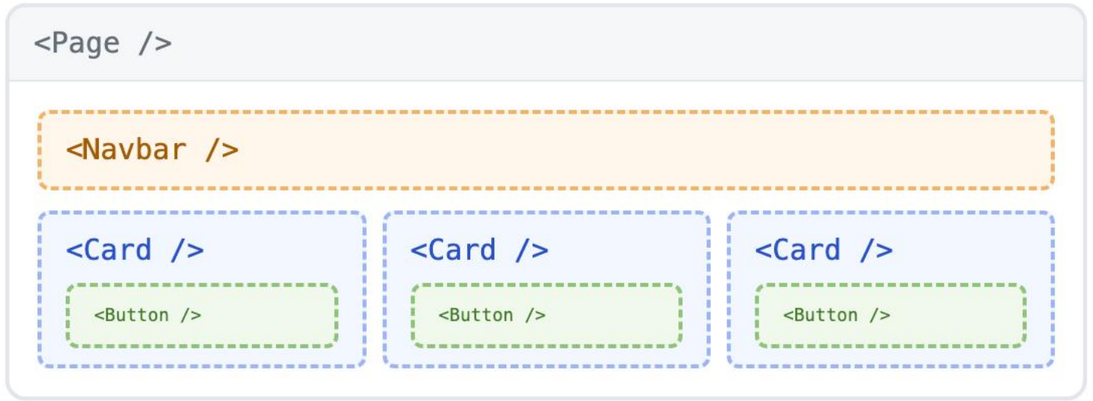
- Updating the page by hand every time data changed
- Copy-pasting the same UI over and over
- No reuse: every button rebuilt from scratch
- Hard to scale as apps grew

Components

- Small, reusable pieces of UI
- Build a Card once, use it 50 times
- React made this the standard

Used by





React: components and JSX

```
function Card() {
  return (
    <div className="card">
      <h2>Maya</h2>
      <p>React, Figma</p>
    </div>
  );
}

<Card />
```

- A component is a function
- It returns JSX: HTML inside JavaScript
- `className`, not `class`
- Use it like a tag: `<Card />`

React: props and lists

```
function Card({ name, skills }) {
  return <h2>{name}: {skills}</h2>;
}

<Card name="Maya" skills="React" />
<Card name="Arjun" skills="Python" />

const people = ["Maya", "Arjun", "Lena"];

people.map((name) => (
  <Card key={name} name={name} />
));
```

- Props pass data into a component
- `{ }` runs JavaScript inside JSX
- `.map()` turns a list into components
- Each item needs a `key`

React: state and events

Plain JavaScript (demo 1)

```
let count = 0;
const button = document.getElementById("btn");

button.addEventListener("click", () => {
  count = count + 1;
  button.textContent = "Clicked " + count + " times";
});
```

React

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

- `useState` holds data that changes
- `onClick` handles the click
- Change the state and React updates the page

The modern toolkit



TypeScript

Types



React

Components



Next.js

Pages + routing



Tailwind CSS

Styling

The AI shift

THEN

Write every line.

NOW

Describe, review, fix.



Pick your platform first

Building	Common stack
Web app or dashboard	Next.js, Tailwind, shadcn/ui, Supabase
Marketing site	Astro or Next.js + a CMS
Mobile app	React Native with Expo, or Flutter
Data or AI demo	Streamlit or Gradio
3D or game	Three.js, React Three Fiber

The stack at a glance

Next.js	Framework
Tailwind CSS	Styling
shadcn/ui	Components
Supabase	Backend
Vercel	Hosting

Next.js and TypeScript

```
app/
  page.tsx              →  /
  dashboard/
    page.tsx            →  /dashboard
  profiles/[id]/
    page.tsx            →  /profiles/42
  api/profiles/
    route.ts            →  /api/profiles
```

- A file is a page
- API routes live in the same project
- TypeScript catches typos early

Tailwind and shadcn/ui

Tailwind

```
<button class="bg-indigo-600 text-white
  px-5 py-2.5 rounded-lg font-semibold">
  Join team
</button>
```

Join team

shadcn/ui

Button | Outline | Dialog | Table

```
npx shadcn@latest add button dialog
```

Supabase and Vercel

Supabase

- Postgres database
- Login
- File storage

Vercel



What is a design system?

Tokens

Colors, fonts, spacing



Components

Buttons, cards, modals

Button Card

Guidelines

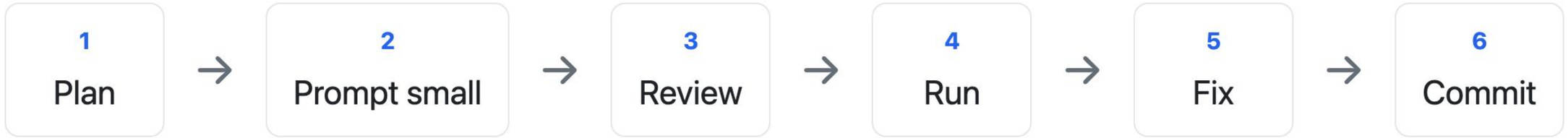
Do / don't

Shopify Polaris Adobe Spectrum Google Material

A design system for your hackathon



The AI workflow



↻ Repeat for every feature

LIVE DEMO

Let's build it

McGill Apartment Finder



AI app builders

Lovable

v0

Bolt.new

Replit Agent

AI code editors

- Claude Code
- Cursor
- GitHub Copilot
- Windsurf

Component libraries

- shadcn/ui
- Magic UI
- Aceternity UI
- MUI
- Mantine
- DaisyUI

Design tools

- Figma
- Figma Make
- Claude Design

Front-end skills for Claude Code

#	Skill	From
1	`frontend-design`	Anthropic
2	`web-design-guidelines`	Vercel
3	`design-taste-frontend`	taste-skill
4	`anti-ui-slop`	GitHub awesome-copilot
5	`ui-ux-pro-max`	nextlevelbuilder
6	`high-end-visual-design`	taste-skill-classics
7	`redesign-existing-projects`	taste-skill

Testing with Playwright

- Tests in a real browser
- Record a flow, rerun it after changes
- MCP: the AI clicks through and fixes bugs



playwright.dev · Playwright MCP

LIVE DEMO

Let's test it

Testing with Playwright



Hackathon tips

- Build the demo path first
- Deploy on day one
- Commit often
- Use real data
- Don't over-polish

Resources + Q&A



Get this deck

- Next.js docs
- shadcn/ui
- Lovable
- Playwright
- Awesome AI Tools for UI
- Tailwind CSS docs
- Supabase docs
- v0
- Anthropic skills
- Design skills ranking